

Case Study: Architecting the "Intent-Based" Developer Platform

Overview and Instructions

This exercise is designed to help us understand your approach to complex platform problems; how you frame them, make assumptions, balance developer needs with infrastructure stability, and communicate a clear plan.

We recommend spending roughly not more than 120 minutes on this take-home portion, which will then serve as a foundation for discussion during the panel interviews. You may submit a short memo or a brief slide deck, depending on your preference.

We know everyone has different working styles. The goal isn't to test speed or presentation polish. What we're looking for is structured thinking, judgment, and a strong product mindset.

AI Note: At _____, we believe in using the best tools to move fast. You are encouraged to use AI (e.g., ChatGPT, Gemini, Cursor, Claude, Mermaid.js etc.) to brainstorm, generate diagrams, or model data. However, your submission must represent your original strategic logic. You must be able to justify the architectural choices during the live discussion.

Please include a brief appendix or screenshot of a few key AI prompts you used during this process. We are interested in how you used AI to accelerate your thinking or troubleshoot a technical trade-off.

Finally, please note that all submissions must be 100% original and must not contain any materials, concepts, or proprietary information belonging to a third party. In the event that you join _____, by providing this submission, you (i) represent that you have unlimited rights to the materials and (ii) grant _____ a perpetual, irrevocable, worldwide, and gratis license to use the submitted materials as determined by _____.

Context

Assume you are a Senior Product Manager on the Platforms team at XYZ Inc. Your mission is to provide the foundational infrastructure that enables every engineer in the company to build, deploy, and scale world-class products.

XYZ Inc's software engineers are highly productive when writing code, but they hit a "wall of toil" when trying to deploy that code to production.

Currently, XYZ's platform consists of several mature but disconnected internal services:

- Provisioning: Setting up cloud resources, clusters, and namespaces.
- Security & Compliance: Managing secrets, IAM roles, and vulnerability scanning.
- Observability: Configuring logs, metrics, and alerting thresholds.
- Traffic Management: Setting up load balancers, DNS, and routing rules.

In the context of the SDLC, these services ensure that, once an application is developed and integrated, it operates in a robust, secure, and feature-rich environment.

The Problem: To launch a simple microservice, a developer must manually "glue" these services together by writing thousands of lines of boilerplate configuration (e.g., YAML, Terraform).

Vision

We want to move from **Manual Configuration** to **Intent-Based Infrastructure**.

Instead of a developer saying, *"I need a Load Balancer with these 20 specific networking rules,"* they should be able to state their intent: *"I am deploying a Tier-2 Python service that needs to be accessible in North America and handle 1k RPS."* An **AI-Agentic Layer** should then interpret that intent, interface with our underlying systems, and generate the necessary configurations automatically.

Constraints

As you build your plan, keep the following real-world constraints in mind:

- **Resource Focus:** You have one dedicated engineering squad with 4-5 engineers for the next quarter. You cannot fix everything at once; you must choose where to start.
- **The Trust Gap:** Many senior engineers at XYZ Inc. have built their own "workarounds" over the years and are naturally skeptical of automated or "AI-managed" infrastructure.
- **Security & Compliance:** Any automation must adhere to strict global security protocols. A "hallucination" in an infrastructure configuration can result in a production outage.

Your Task

As the Lead PM for the Developer Experience (DevEx) team, you are tasked with defining the roadmap for this AI-Agentic Layer.

Create a strategic proposal for how you would lead the transition from manual configuration to an AI-Agentic Platform over the next 6 months.

We are not looking for a perfect technical architecture; rather, we want to see your strategic thinking. In your response, please address:

Part 1: Problem Discovery and Framing

- How do you determine which part of the path to production is the most broken and causing the most pain for customers?
- What data or signals would you use to quantify this toil; and what would success look like for our developers? (Frame and share your assumptions if any)
- How would you validate your assumptions with our internal engineers without being a distraction to their sprint? If this AI layer is successful, how will the organization change? Define at least 2–3 North Star metrics.

Part 2: Strategy and Solutioning

- Propose & describe a single interface or agentic flow that acts as a translation layer for the customer's intent (You can choose and mention the layers you are targeting and assumptions you are making, but justify your choice)
- How would you prioritize which phase of the lifecycle or service to automate first? Show us the logic behind what you choose to build vs. what you choose to delay.
- How do you connect developer value (speed) with business results (stability/cost)?
- (Optional) - Use an AI tool (v0, Mermaid, etc.) to show us the new developer interface. How does the solution change the developer's "Tuesday afternoon"?

Part 3: Practical Judgment and Adoption

- How do you move from a vision to a testable MVP? What is the quickest way to learn if your solution actually solves the problem? Describe a MVP that could provide value to a single team within 4 weeks.
- How do you build trust with skeptical internal customers? How do you design the product to provide transparency and control so senior engineers trust the AI?
- What is your approach to marketing this platform internally to ensure adoption?